

# Aula 02

Prof. Dr. Alencar Machado

# Agenda

- Boas práticas API Rest
  - Validação
  - Respostas de operações
  - Tratamento de erros
- Spring Data JPA
  - One-to-Many
  - Many-to-One

# Spring Validation

JSR 380 implementation — Hibernate Validator

- @NotBlank
- @Size
- @Email
- @NotNull
- @Pattern
- @Valid

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-validation</artifactId>  
</dependency>
```

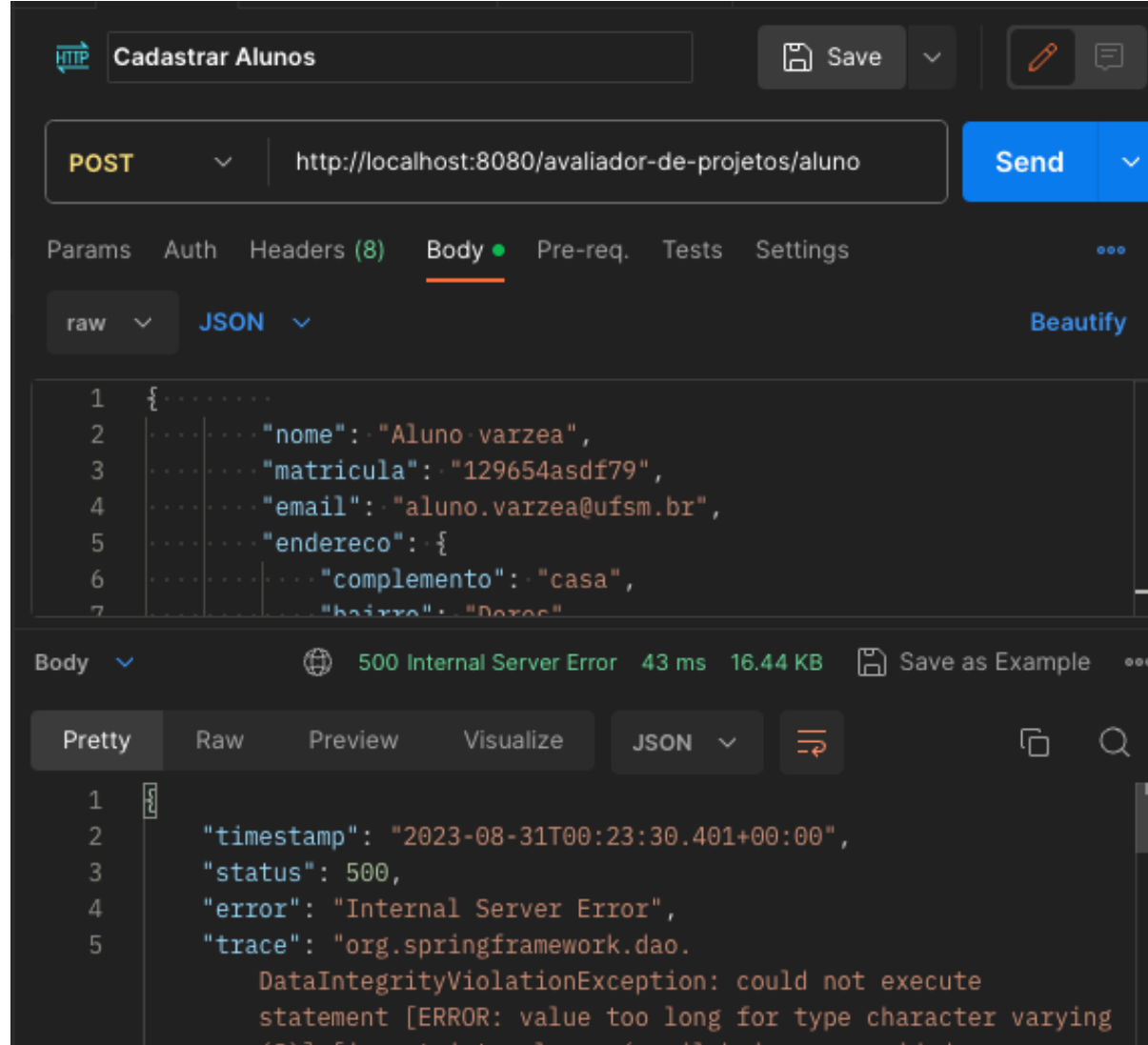
# Email inválido

The screenshot shows a REST client interface with a POST request to `http://localhost:8080/avaliador-de-projetos/aluno`. The request body is a JSON object with the following fields:

```
{  "nome": "Aluno email invalido",  "matricula": "12998788",  "email": "email invalido",  "endereco": {    "complemento": "casa",    "bairro": "Dores",    "cep": "97050120",    "numero": "1345",    "cidade": "Brasilia",    "uf": "DF"  },}
```

The response status is `200 OK` with a response time of `58 ms` and a body size of `123 B`. The interface includes tabs for Params, Auth, Headers (8), Body (selected), Pre-req., Tests, and Settings. The Body tab is set to JSON format. The bottom bar shows the 'Pretty' view selected, along with icons for copy, search, and other utilities.

# Matricula > 8 caracteres



```
org.postgresql.util.PSQLException Create breakpoint : ERROR: value too long for type character varying(8)
at org.postgresql.core.v3.QueryExecutorImpl.receiveErrorResponse(QueryExecutorImpl.java:2713) ~[postgr
at org.postgresql.core.v3.QueryExecutorImpl.processResults(QueryExecutorImpl.java:2401) ~[postgresql-4
```

```

public class Aluno {

    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @NotBlank
    private String nome;

    @Size(max = 8, message = "Matricula deve ter no máximo 8 digit
    private String matricula;
    @Email(message = "Email inválido")
    private String email;

    @Embedded
    private Endereco endereco;

}

```

```

public class Endereco {

    private String complemento;
    @NotBlank
    private String bairro;
    @Size(min = 8, max = 9, message = "CEP inválido")
    private String cep;
    @NotBlank
    private String numero;
    @NotBlank
    private String cidade;
    @NotBlank
    @Size(max = 2)
    private String uf;

}

```



```

@PostMapping
@Transactional
public void salvar(@RequestBody @Valid Aluno aluno) { this.service.salvar(aluno); }

```

POST

http://localhost:8080/avaliador-de-projetos/aluno

Send

Params Auth Headers (8) Body Pre-req. Tests Settings

raw

JSON

Beautify

```
1 {
2   "nome": "Aluno email invalido",
3   "matricula": "12998788",
4   "email": "email invalido",
```

Body



400 Bad Request

318 ms

724 B



Save as Example



Pretty

Raw

Preview

Visualize

JSON



```
1 {
2   "timestamp": "2023-08-31T00:50:59.915+00:00",
3   "status": 400,
4   "error": "Bad Request",
5   "message": "Validation failed for object='aluno'. Error
6     count: 1",
7   "errors": [
8     {
9       "codes": [
10        "Email.aluno.email",
11        "Email.email",
12        "Email.java.lang.String",
13        "Email"
14      ],
15      "arguments": [
16        {
17          "codes": [
18            "aluno.email",
19            "email"
20          ],
21          "arguments": null,
22          "defaultMessage": "email",
23          "code": "email"
```

POST

http://localhost:8080/avaliador-de-projetos/aluno

Send

Params Auth Headers (8) Body Pre-req. Tests Settings

raw

JSON

Beautify

```
1 {
2   "nome": "Aluno email invalido",
3   "matricula": "12998789878",
4   "email": "email@gmail.com",
5   "endereco": {
6     "complemento": "casa",
7     "bairro": "Dores",
8     "cep": "97050120",
9     "numero": "1345",
10    "cidade": "Brasilia",
11    "uf": "DF"
12  },
13  "projeto": {
```

Body



400 Bad Request

69 ms

713 B



Save as Example



Pretty

Raw

Preview

Visualize

JSON



```
19   },
20   "arguments": null,
21   "defaultMessage": "matricula",
22   "code": "matricula"
23 },
24 8,
25 0
26 ],
27 "defaultMessage": "Matricula deve ter no máximo 8
28   digitos",
29 "objectName": "aluno",
30 "field": "matricula",
31 "rejectedValue": "12998789878",
32 "bindingFailure": false,
33 "code": "Size"
34 ],
35 "path": "/avaliador-de-projetos/aluno"
```

# Retornos REST

ResponseEntity

# REST: Oq Podemos melhorar?

- Estamos utilizando os verbos HTTP?
- Como está o tratamento de retorno das requisições da API?

DELETE



http://localhost:8080/avaliador-de-projetos/aluno/7

Send



Params

Auth

Headers (6)

Body

Pre-req.

Tests

Settings

Cookies

Query Params

|  | Key | Value | Description | ⋮ Bulk Edit |
|--|-----|-------|-------------|-------------|
|--|-----|-------|-------------|-------------|

Body



200 OK

79 ms

123 B



Save as Example



Pretty

Raw

Preview

Visualize

Text



1

200, indica sucesso da solicitação

PUT

http://localhost:8080/avaliador-de-projetos/aluno

Send

Params

Auth

Headers (8)

Body

Pre-req.

Tests

Settings

Cookies

raw

JSON

Beautify

1

2

3

4

5

6

7

8

"id": 6,

"nome": "Novo",

"matricula": "12993392",

"email": "atualizar@ufsm.br",

"endereco": {

"complemento": "casa",

"bairro": "Dores",

Body

200 OK

117 ms

123 B

Save as Example

200 NOVAMENTE ... será q só devemos retornar 200 ?

Pretty

Raw

Preview

Visualize

Text

1

# Delete: 204 Requisição OK e não tem um corpo de retorno

```
alencar.machado
@DeleteMapping("/{id}")
public void deletar(@PathVariable Long id){
    this.service.excluir(id);
}
```

```
alencar.machado *
@DeleteMapping("/{id}")
public ResponseEntity deletar(@PathVariable Long id){
    this.service.excluir(id);
    return ResponseEntity.noContent().build();
}
```

DELETE http://localhost:8080/avaliador-de-projetos/aluno/6

Params Auth Headers (6) Body Pre-req. Tests Settings Cookies

Query Params

| Key | Value | Description | ... | Bulk Edit |
|-----|-------|-------------|-----|-----------|
|-----|-------|-------------|-----|-----------|

Body 204 No Content 264 ms 112 B Save as Example

Pretty Raw Preview Visualize Text

1

# PUT

```
@PutMapping
@Transactional
public ResponseEntity atualizar(@RequestBody Aluno aluno){
    this.service.atualizar(aluno);
    return ResponseEntity.ok(aluno);
}
```

`ResponseEntity.ok().body(aluno)`

The screenshot displays a REST client interface for a PUT request. The URL is `http://localhost:8080/avaliador-de-projetos/aluno`. The request body is a JSON object with the following fields: `id` (8), `nome` ("Aluno Atualizado"), `matricula` ("78945612"), and `email` ("atualizado@email0asd"). The response status is 200 OK, with a response time of 451 ms and a body size of 381 B. The response body is a JSON object with the following fields: `id` (8), `nome` ("Aluno Atualizado"), `matricula` ("78945612"), `email` ("atualizado@email0asd"), and `endereco` (an object with `complemento` ("casa"), `bairro` ("Dores"), `cep` ("97050120"), `numero` ("1345"), `cidade` ("Brasilia"), and `uf` ("DF")).

```
PUT http://localhost:8080/avaliador-de-projetos/aluno
```

Params Auth Headers (8) Body Pre-req. Tests Settings

raw JSON Beautify

```
1 {
2   "id": 8,
3   "nome": "Aluno Atualizado",
4   "matricula": "78945612",
5   "email": "atualizado@email0asd",
```

Body 200 OK 451 ms 381 B Save as Example

Raw Preview Visualize JSON

```
{
  "id": 8,
  "nome": "Aluno Atualizado",
  "matricula": "78945612",
  "email": "atualizado@email0asd",
  "endereco": {
    "complemento": "casa",
    "bairro": "Dores",
    "cep": "97050120",
    "numero": "1345",
    "cidade": "Brasilia",
    "uf": "DF"
  },
}
```

# Criar/Salvar: Dever ser retornado 201

- 201: Created

## Código 201

devolve no corpo da resposta os dados do novo  
recurso/registro criado

## Código 201

devolve também um cabeçalho do protocolo HTTP  
(Location)

# Refatorando o Salvar.

```
alencar.machado
@PostMapping
@Transactional
public void salvar(@RequestBody @Valid Aluno aluno){
    this.service.salvar(aluno);
}
```

```
alencar.machado *
@PostMapping
@Transactional
public ResponseEntity salvar(@RequestBody @Valid Aluno aluno, UriComponentsBuilder uriBuilder){

    this.service.salvar(aluno);
    //monta a URI da aplicação dinamicamente
    URI uri = uriBuilder.path("/aluno/{id}").buildAndExpand(aluno.getId()).toUri();
    //created(uri) irá colocar no cabeçalho da requisição da resposta
    // o parâmetro Location com a URI de acesso ao recurso criado
    return ResponseEntity.created(uri).body(aluno);
}
```

POST

http://localhost:8080/avaliador-de-projetos/aluno

Send

Params Auth Headers (8) Body Pre-req. Tests Settings

Cookies

raw

JSON

Beautify

```
1 {
2   "nome": "Tais",
3   "matricula": "12365478",
4   "email": "tais@gmail.com",
5   "endereco": {
6     "complemento": "casa",
7     "bairro": "Dores",
8     "cep": "97050120",
9     "numero": "1345",
10    "cidade": "Brasilia",
11    "uf": "DF"
12  },
```

alencar.machado

```
@GetMapping("/{id}")
public Aluno aluno(@PathVariable Long id){
    return this.service.findById(id);
}
```

Headers



201 Created

97 ms

558 B



Save as Example



| Key               |  | Value                                                |
|-------------------|--|------------------------------------------------------|
| Location          |  | http://localhost:8080/avaliador-de-projetos/aluno/20 |
| Content-Type      |  | application/json                                     |
| Transfer-Encoding |  | chunked                                              |
| Date              |  | Thu, 31 Aug 2023 01:06:20 GMT                        |
| Keep-Alive        |  | timeout=60                                           |
| Connection        |  | keep-alive                                           |

# Tratamento de erros

# Tratamento de erros

Alunos / Obter Aluno

GET <http://localhost:8080/avaliador-de-projetos/aluno/1> Send

Params Auth Headers (6) Body Pre-req. Tests Settings Cookies

Query Params

| Key | Value | Descript... | ... | Bulk Edit |
|-----|-------|-------------|-----|-----------|
|-----|-------|-------------|-----|-----------|

Body 200 OK 58 ms 492 B Save as Example

Pretty Raw Preview Visualize JSON

```
1 {
2   "id": 1,
3   "nome": "Alencar",
4   "matricula": "12993399",
5   "email": "alencar@ufsm.br",
6   "endereco": {
7     "complemento": "casa",
8     "bairro": "Dores",
9     "cep": "97050120",
10    "numero": "1345",
11    "cidade": "Brasilia",
12    "uf": "DF"
13  },
14  "projeto": {
15    "id": 1,
16    "nome": "Projeto Casa da Colina",
17    "descricao": "Projeto da familia em voltar para a casa da colina",
18    "ano": 2023,
19    "semestre": 2
20  }
21 }
```

OK

Alunos / Obter Aluno

GET <http://localhost:8080/avaliador-de-projetos/aluno/asdf> Send

Params Auth Headers (6) Body Pre-req. Tests Settings Cookies

Query Params

| Key | Value | Descript... | ... | Bulk Edit |
|-----|-------|-------------|-----|-----------|
|-----|-------|-------------|-----|-----------|

Body 400 Bad Request 217 ms 6.46 KB Save as Example

Pretty Raw Preview Visualize JSON

```
1 {
2   "timestamp": "2023-08-30T22:47:20.671+00:00",
3   "status": 400,
4   "error": "Bad Request",
5   "trace": "org.springframework.web.method.annotation.
      MethodArgumentTypeMismatchException: Failed to convert value of
      type 'java.lang.String' to required type 'java.lang.Long'; For
      input string: \"asdf\"\\n\\tat org.springframework.web.method.
```

Bad Request

aluno/asdf  
aluno/99

Alunos / Obter Aluno

GET <http://localhost:8080/avaliador-de-projetos/aluno/99> Send

Params Auth Headers (6) Body Pre-req. Tests Settings Cookies

Query Params

| Key | Value | Descript... | ... | Bulk Edit |
|-----|-------|-------------|-----|-----------|
|-----|-------|-------------|-----|-----------|

Body 500 Internal Server Error 88 ms 5.96 KB Save as Example

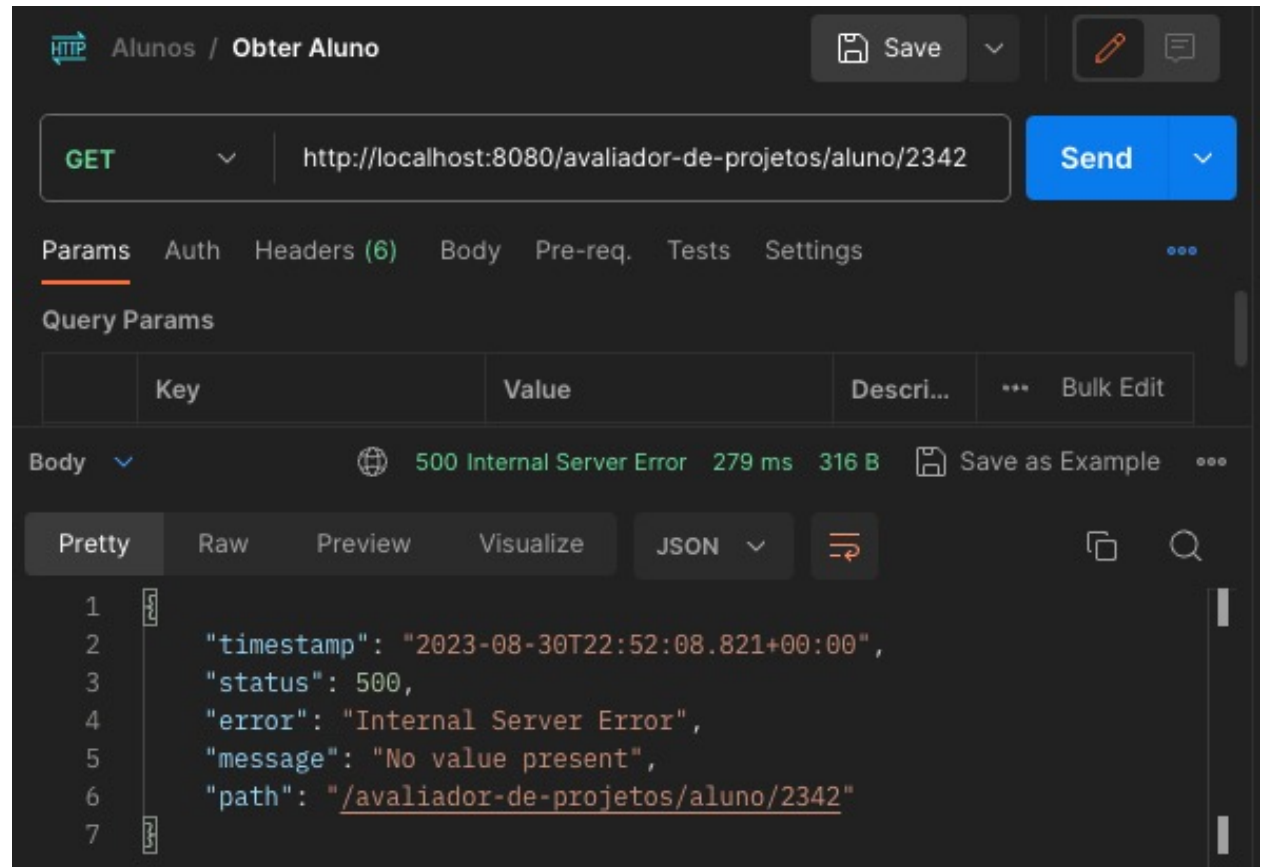
Pretty Raw Preview Visualize JSON

```
1 {
2   "timestamp": "2023-08-30T22:47:50.176+00:00",
3   "status": 500,
4   "error": "Internal Server Error",
5   "trace": "java.util.NoSuchElementException: No value present\\n\\tat
      java.base/java.util.Optional.get(Optional.java:143)\\n\\tat br.csi.
      avaliadordeprojetos.service.AlunoService.findById(AlunoService.
      java:24)\\n\\tat br.csi.avaliadordeprojetos.controller.
      AlunoController.aluno(AlunoController.java:32)\\n\\tat java.base/
      jdk.internal.reflect.NativeMethodAccessorImpl.invoke0(Native
```

Internal Server Error

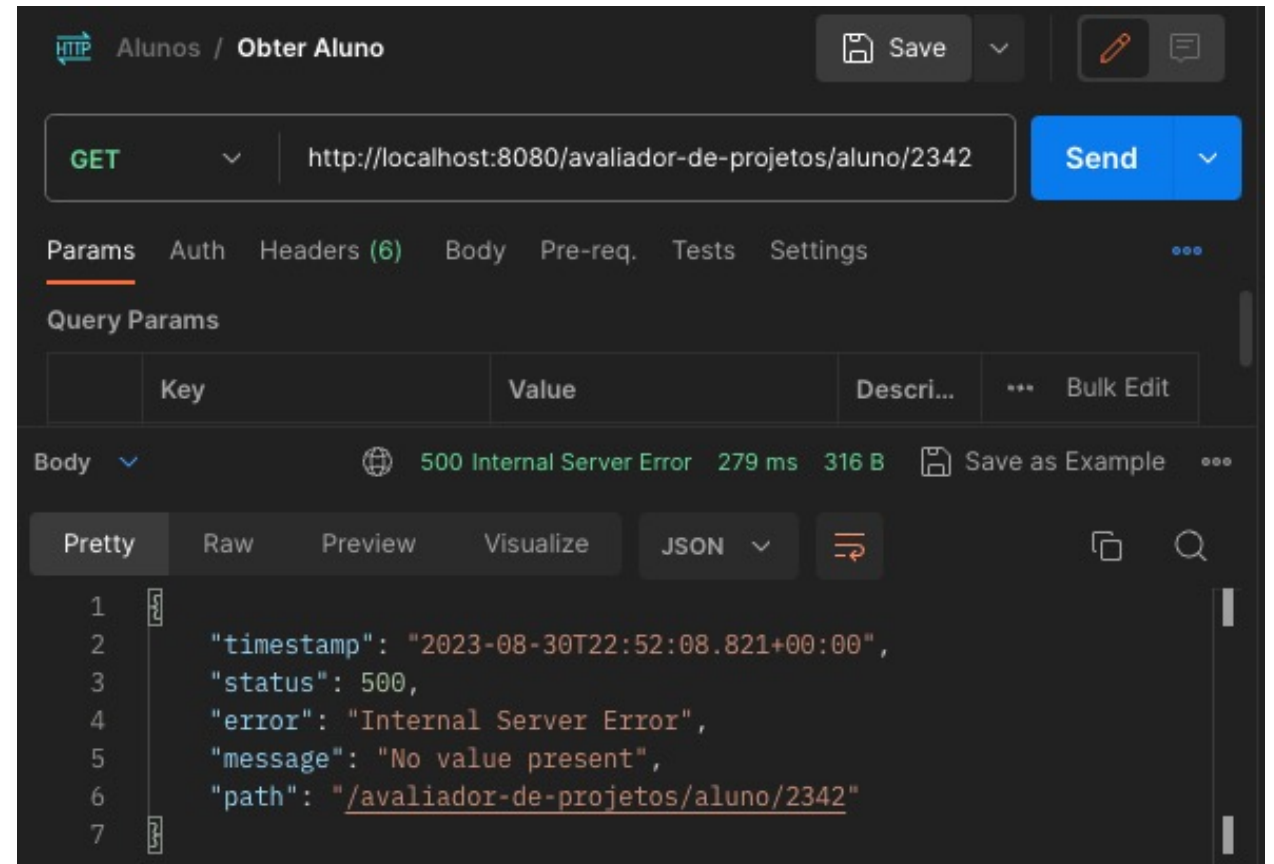
# Tratamento de erros

- Desabilitar o trace no application.properties
  - `server.error.include-stacktrace=never`



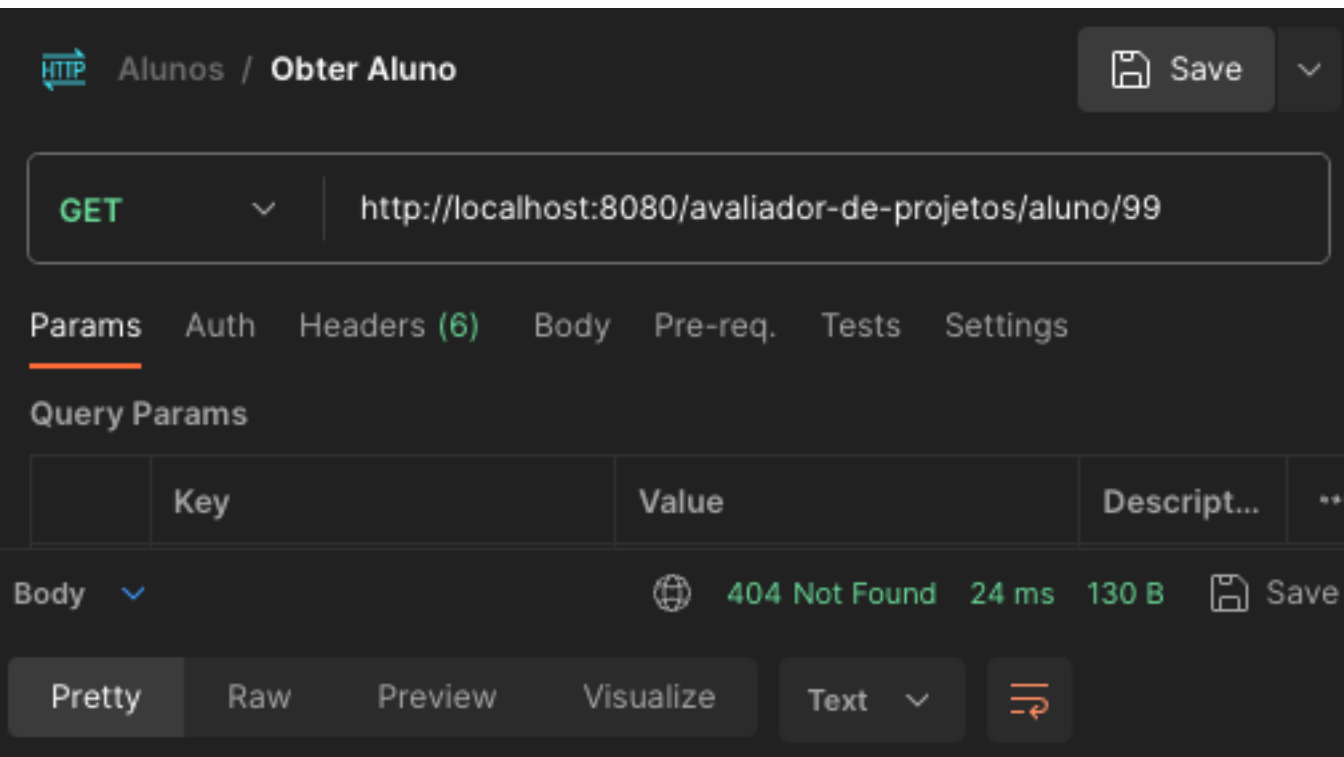
# Tratamento de erros

- Estamos tentando acessar um ID q não existe ... deveríamos receber um 404, certo?
- Padrão do spring data é 500



# Recurso não encontrado: 404

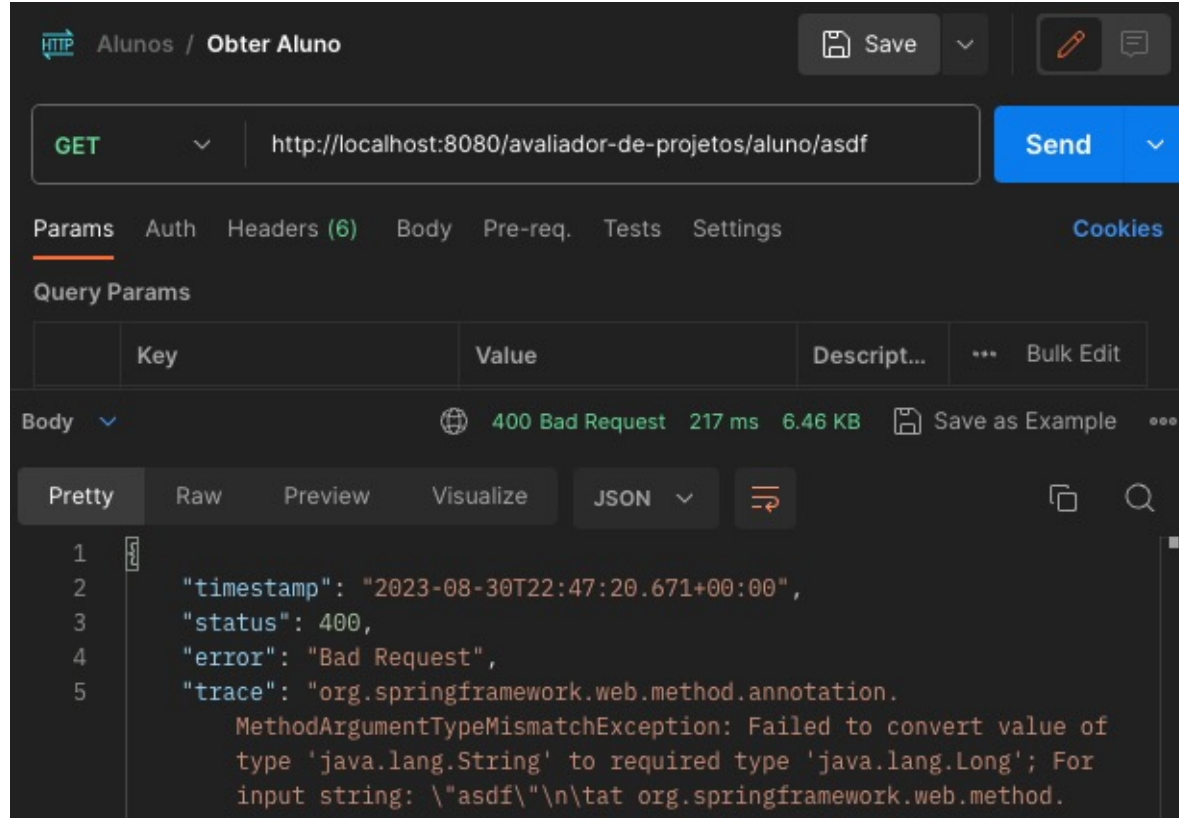
- Criar package infra e dentro deste a classe TratadorDeErros



```
new *
@RestControllerAdvice
public class TratadorDeErros {
    new *
    @ExceptionHandler(NoSuchElementException.class)
    public ResponseEntity tratarErro404() {
        return ResponseEntity.notFound().build();
    }
}
```

# Persolnização de exceptions

- Como tratar 400 Bad Request?



The screenshot shows a web browser interface with a URL bar displaying `http://localhost:8080/avaliador-de-projetos/aluno/asdf`. The browser has tabs for "Alunos" and "Obter Aluno". Below the URL bar, there are tabs for "Params", "Auth", "Headers (6)", "Body", "Pre-req.", "Tests", "Settings", and "Cookies". The "Body" tab is selected, showing a "400 Bad Request" error. The error message is displayed in a "Pretty" view, showing a JSON response with a "timestamp", "status", "error", and "trace" field. The "trace" field contains a detailed message about a `MethodArgumentTypeMismatchException` occurring during a validation process.

```
1 {
2   "timestamp": "2023-08-30T22:47:20.671+00:00",
3   "status": 400,
4   "error": "Bad Request",
5   "trace": "org.springframework.web.method.annotation."
```

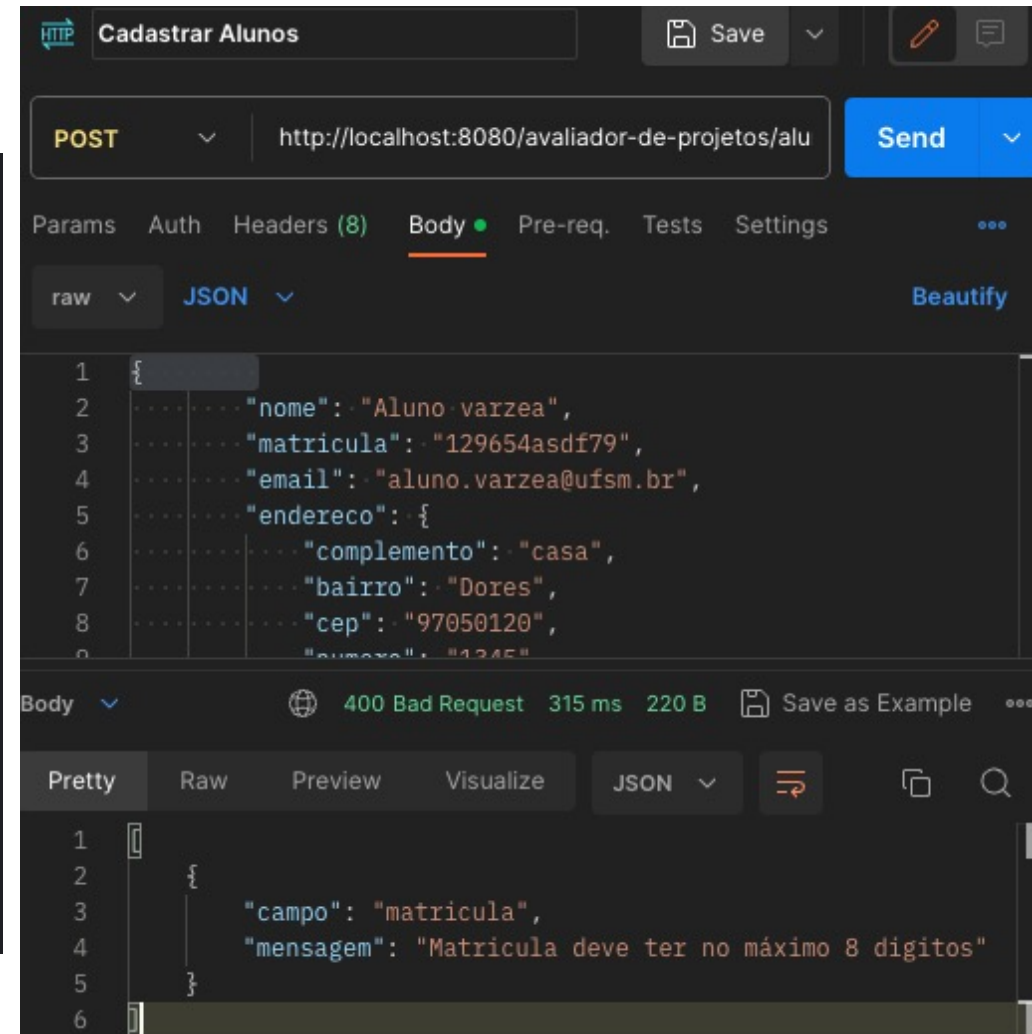
```
"timestamp": "2023-08-30T23:13:50.105+00:00",
"status": 400,
"error": "Bad Request",
"message": "Validation failed for object='aluno'. Error count: 1",
"errors": [
  {
    "codes": [
      "Size.aluno.matricula",
      "Size.matricula",
      "Size.java.lang.String",
      "Size"
    ],
    "arguments": [
      {
        "codes": [
          "aluno.matricula",
          "matricula"
        ],
        "arguments": null,
        "defaultMessage": "matricula",
        "code": "matricula"
      },
      8,
      0
    ],
    "defaultMessage": "Matricula deve ter no máximo 8 digitos",
    "objectName": "aluno",
    "field": "matricula",
    "rejectedValue": "129654asdf79",
    "bindingFailure": false,
    "code": "Size"
  }
]
```

# Persolnização de exceptions

```
//Erro para dados inválidos
new *
@ExceptionHandler(MethodArgumentNotValidException.class)
public ResponseEntity tratarErro400(MethodArgumentNotValidException ex){

    List<FieldError> erros = ex.getFieldErrors();
    List<DadosErroValidacao> dados = new ArrayList<>();
    for(FieldError fe: erros){
        dados.add(new DadosErroValidacao(fe.getField(), fe.getDefaultMessage()));
    }
    return ResponseEntity.badRequest().body(dados);
    //return ResponseEntity.badRequest().body(erros.stream().map(DadosErroValidacao::new).toList());
}

2 usages new *
private record DadosErroValidacao(String campo, String mensagem){
    no usages new *
    public DadosErroValidacao(FieldError erro){
        this(erro.getField(), erro.getDefaultMessage());
    }
}
```



# LifeCycle de Entidades

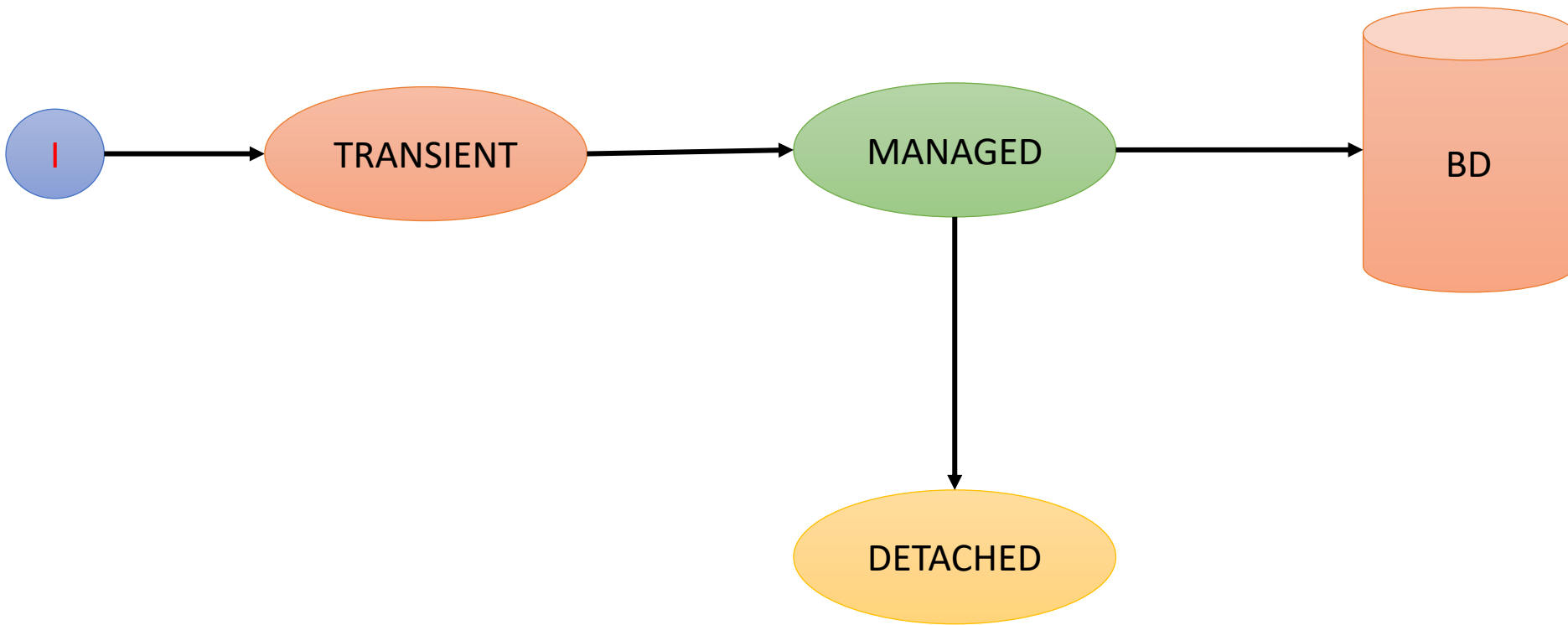
# LifeCycle de Entidades

```
alencar.machado *
@PostMapping
@Transactional
public ResponseEntity salvar(@RequestBody @Valid Aluno aluno, UriComponentsBuilder uriBuilder){

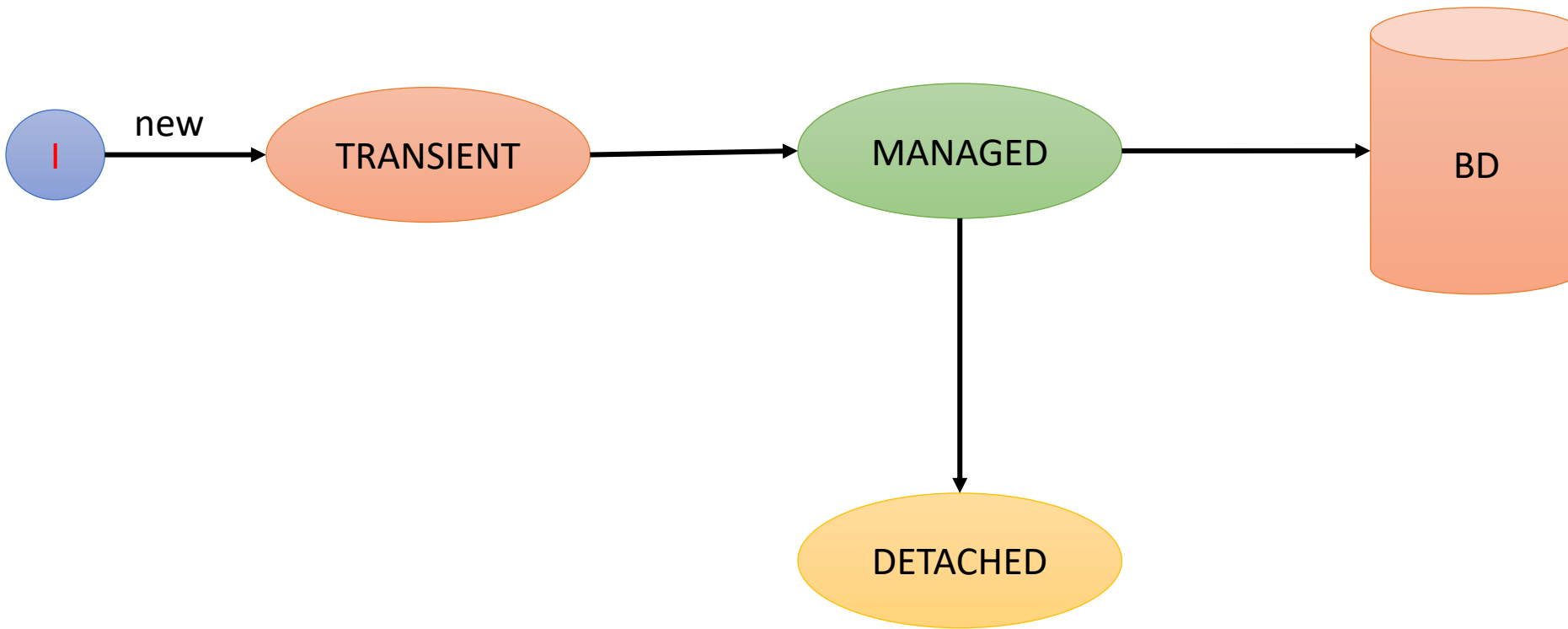
    this.service.salvar(aluno);
    //monta a URI da aplicação dinamicamente
    URI uri = uriBuilder.path("/aluno/{id}").buildAndExpand(aluno.getId()).toUri();
    //created(uri) irá colocar no cabeçalho da requisição da resposta
    // o parâmetro Location com a URI de acesso ao recurso criado
    return ResponseEntity.created(uri).body(aluno);
}
```

Como foi possível ter ID nessa instância?

# LifeCycle de Entidades

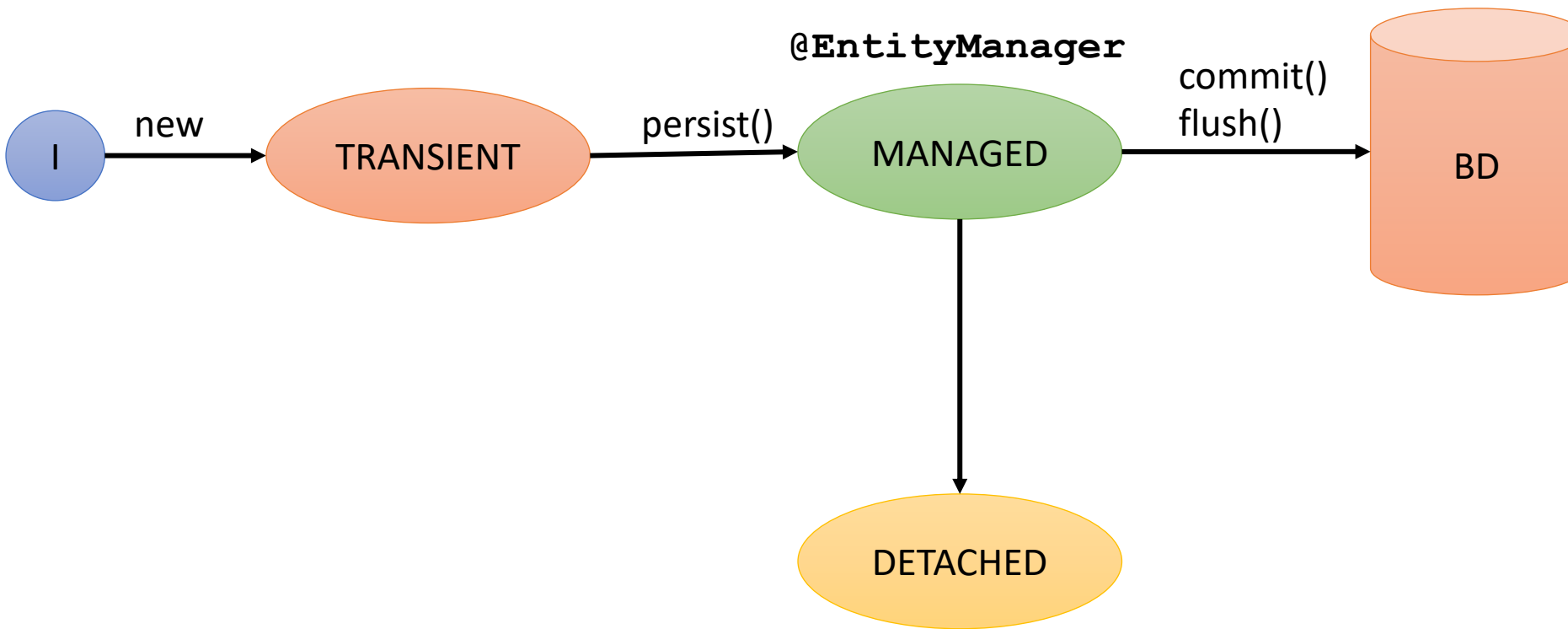


# LifeCycle de Entidades (insert)



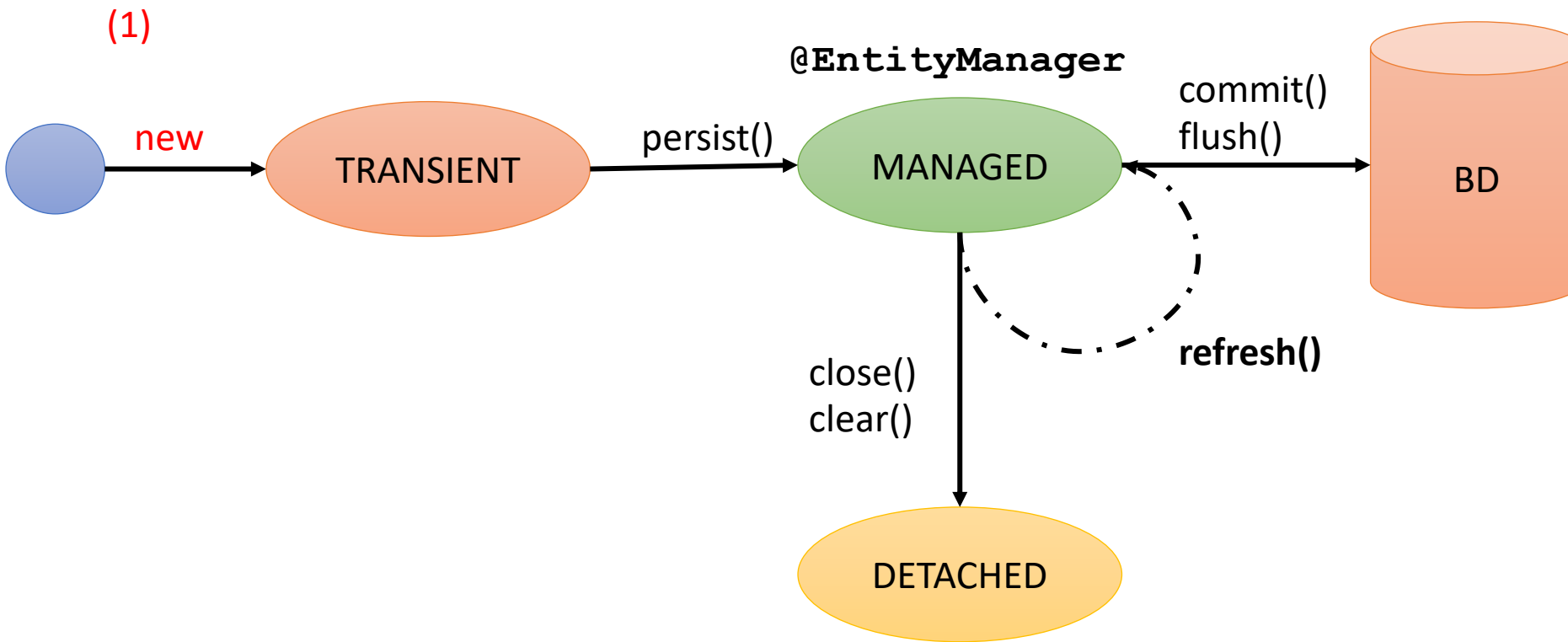
Uma entidade está no estado **transient** quando nunca foi persistida, não está gravada no banco de dados, não tem um *id* e não está sendo gerenciada pela JPA

# LifeCycle de Entidades (insert)



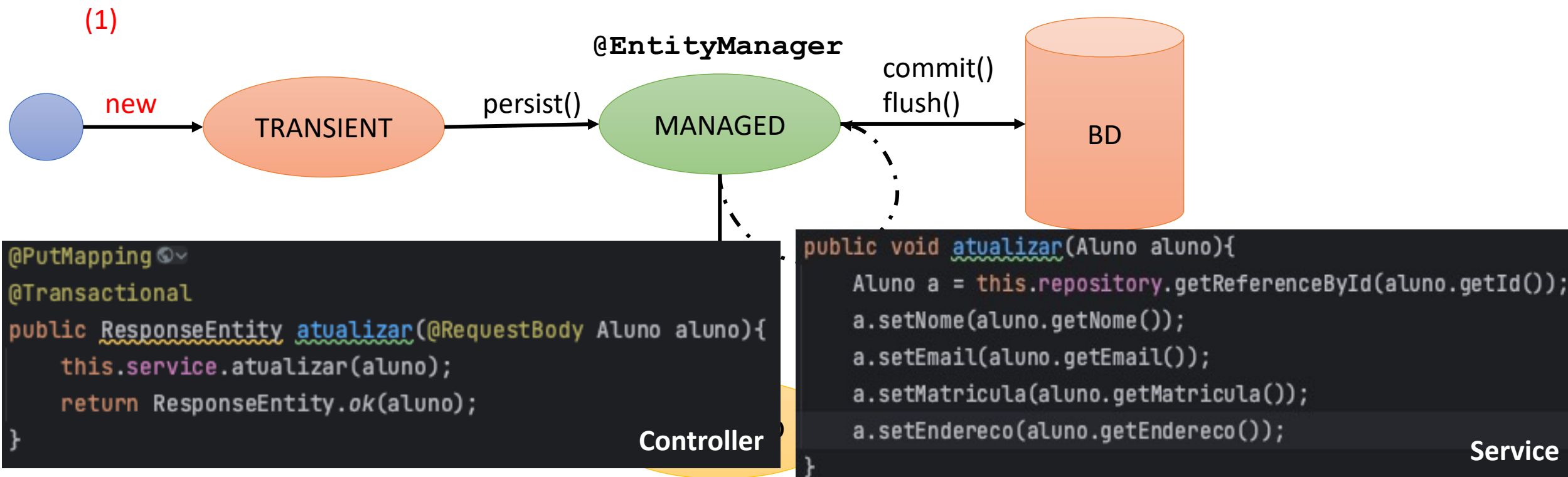
Uma entidade está no estado de **Managed** após ter sido persistida no banco, a partir desse momento tudo que acontecer com esta entidade nesse estado, a JPA observará e poderá sincronizar com o banco de dados. Aqui todos os dados da entidade estarão sincronizados com o banco de dados, portanto se alterarmos o valor de algum atributo, esse valor será atualizado no banco

# LifeCycle de Entidades (update)



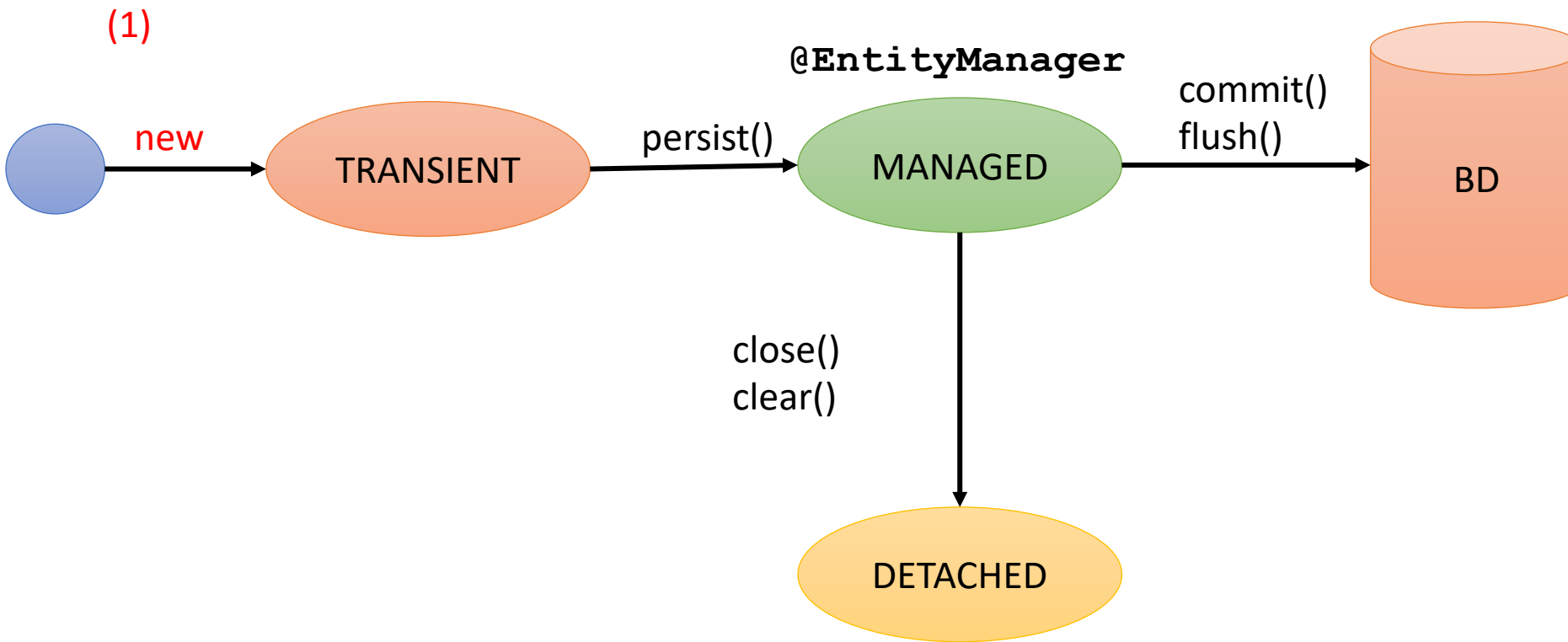
Toda vez que a entidade está no estado managed, está gerenciada, qualquer mudança que fizermos em algum atributo, a JPA detectará e, no **commit()** ou no **flush()** do **EntityManager**, ela vai, automaticamente, sincronizar essas mudanças no banco de dados, porque sabe que é necessário fazer o *update* no banco de dados.

# LifeCycle de Entidades (update)



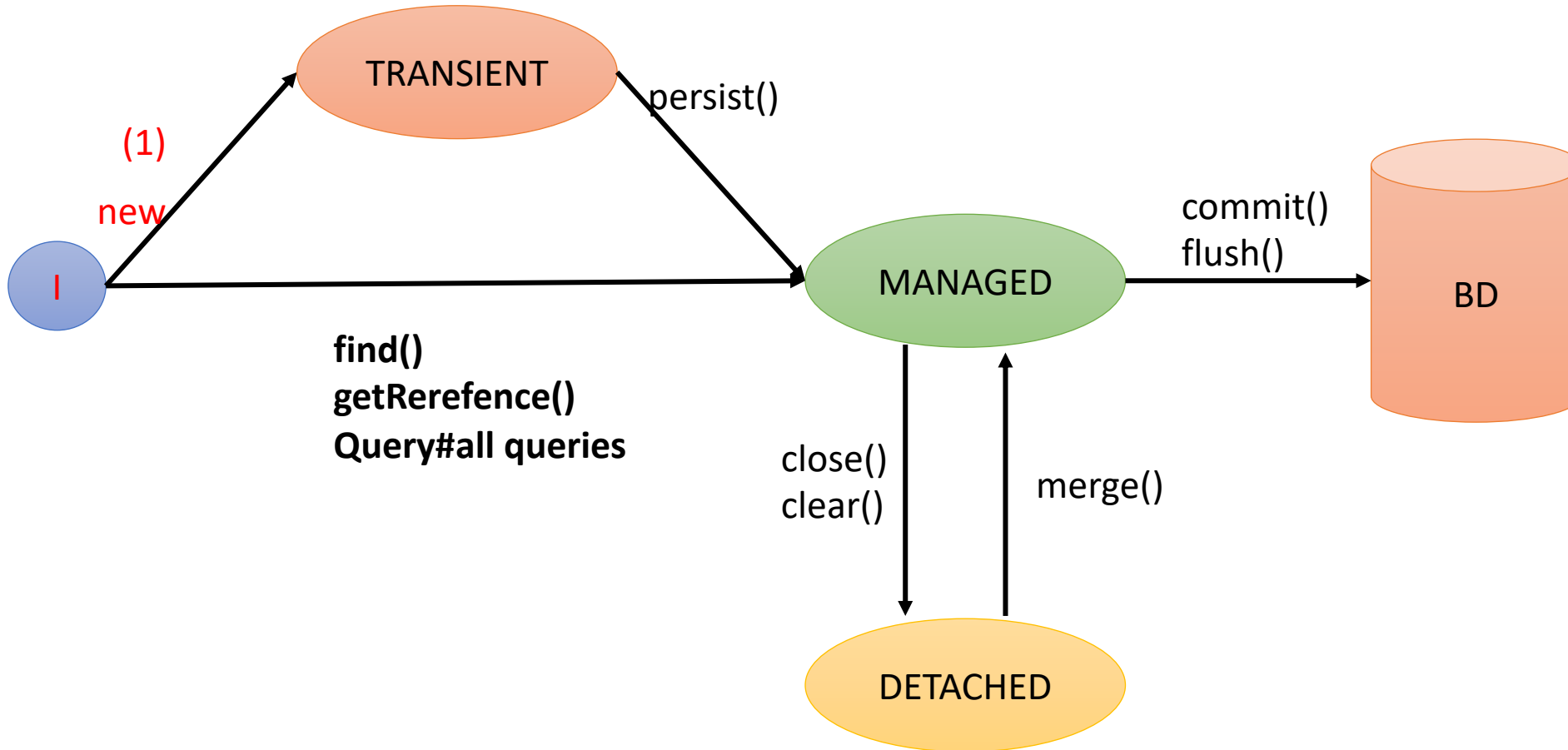
Toda vez que a entidade está no estado managed, está gerenciada, qualquer mudança que fizermos em algum atributo, a JPA detectará e, no `commit()` ou no `flush()` do `EntityManager`, ela vai, automaticamente, sincronizar essas mudanças no banco de dados, porque sabe que é necessário fazer o *update* no banco de dados.

# LifeCycle de Entidades

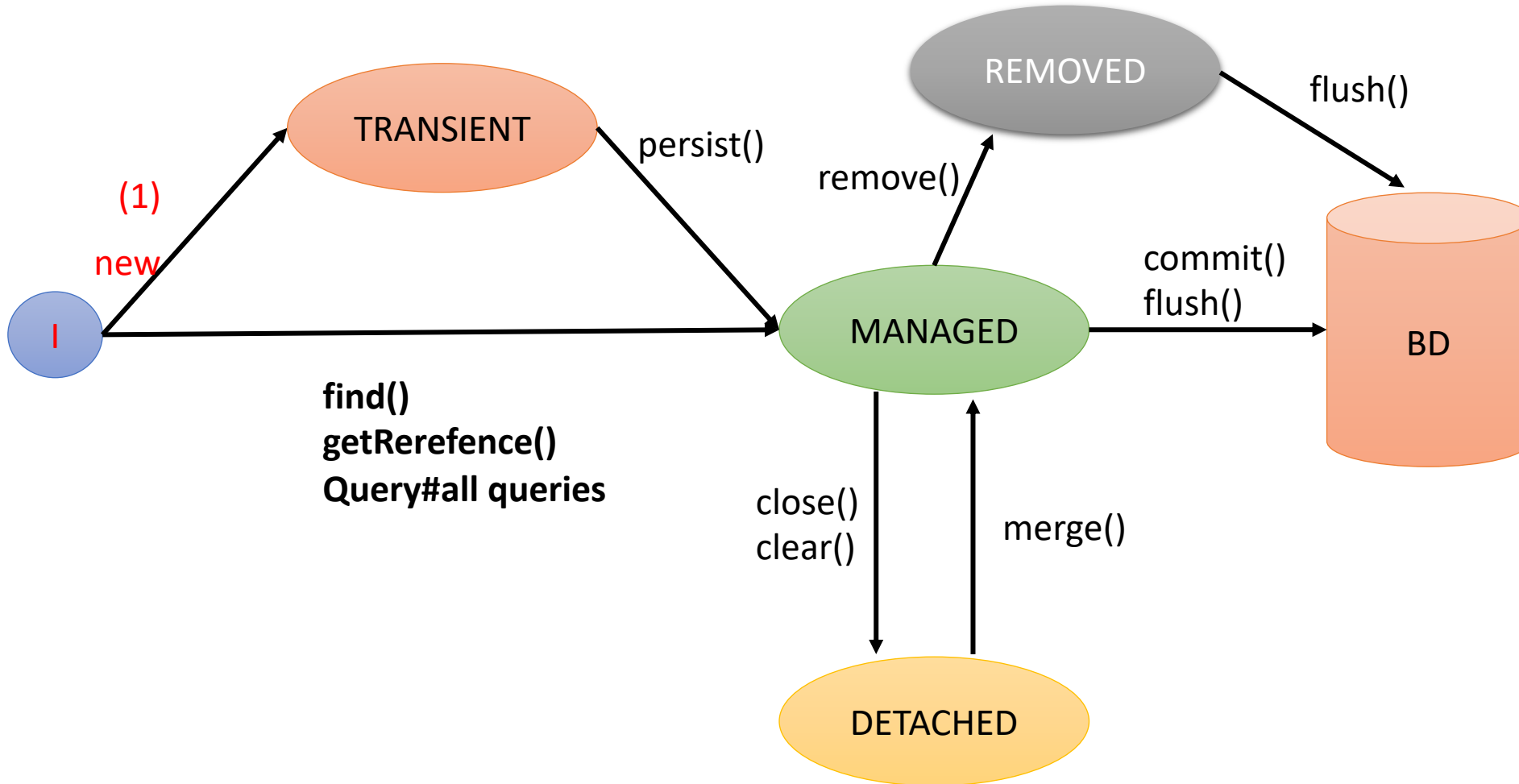


O **detached** é um estado em que a entidade não é mais **transient**, porque tem id, já foi salva no banco de dados, porém, não está mais sendo gerenciada. Portanto, se mexermos nos atributos, a JPA não disparará *update* e nem fará mais nada.

# LifeCycle de Entidades (obter)



# LifeCycle de Entidades (removed)



# OneToMany - ManyToOne

Criar entidade Projeto

# Migration

- V2\_\_update-aluno-create-projetos.sql

```
CREATE TABLE projetos(
    id serial not null primary key,
    nome varchar(100) not null ,
    descricao text not null,
    ano numeric(4,0) not null ,
    semestre numeric(1,0) not null
);

ALTER TABLE alunos ADD COLUMN idprojeto integer;
ALTER TABLE alunos ADD CONSTRAINT aluno_projeto_fkey
    FOREIGN KEY (idprojeto) REFERENCES projetos(id);
```

```
@Entity
@Table(name = "projetos")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
public class Projeto{
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String nome;
    private String descricao;
    private int ano;
    private int semestre;

    @OneToMany(mappedBy = "projeto")
    @JsonIgnore
    private List<Aluno> alunos;
}
```

```
@Entity
@Table(name = "alunos")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
public class Aluno {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @NotBlank
    private String nome;
    @Size(max = 8, message = "Matricula deve ter no máximo 8 dígitos")
    private String matricula;
    @Email(message = "Email inválido")
    private String email;
    @Embedded
    private Endereco endereco;

    @ManyToOne
    @JoinColumn(name = "idprojeto")
    private Projeto projeto;
}
```

# Atividade: Fazer o CRUD de Projetos

- salvar/criar
- listar

# Ajustes: Cadastre alguns projetos

## AlunoService

```
1 usage new *  
public String atribuirProjeto(Long idAluno, Projeto projeto){  
    Aluno aluno = this.repository.getReferenceById(idAluno);  
    aluno.setProjeto(projeto);  
    return "Projeto atribuído com sucesso";  
}
```

## AlunoController

```
@PutMapping("/{id}/atribuir-projeto")  
@Transactional  
public ResponseEntity vincularProjeto(@PathVariable Long id, @RequestBody Projeto projeto){  
    return ResponseEntity.ok(this.service.atribuirProjeto(id, projeto));  
}
```

# @Query

```
public interface AlunoDTO {  
    new *  
    Long getId();  
    no usages new *  
    String getNome();  
    no usages new *  
    String getMatricula();  
    no usages new *  
    String getEmail();  
}
```

```
//https://docs.spring.io/spring-data/jpa/docs/current/reference/html/#projections  
1 usage new *  
@Query(value = "SELECT a.id as id, a.nome as nome, a.matricula as matricula, a.email as email" +  
    " FROM alunos a where a.idprojeto =:id", nativeQuery = true)  
List<AlunoDTO> findAlunosByProjeto(@Param("id") int id);
```

# DTO: Objeto de Transferência de Dados

- Não possui comportamentos;
- Função é obter dados somente (manter o estado do objeto)
- Filtrar quais dados desejamos transmitir do objeto
  - Não expõem todos os dados da classe